Robust Human Activity Recognition Across Smartphones via Unsupervised Per-Device Standardization

Team TEAM_NN: Member 1, Member 2, Member 3, Member 4
Computer Engineering Department, Faculty of Engineering, Cairo University

***Abstract*—We classify smartphone inertial telemetry (561 time- and frequency-domain features) into six activities when the test phones differ from the training phone. The released data contain a single labelled device; the validation set holds two devices whose sensors differ by per-feature calibration offsets and gains, and the feature set includes a block of noise-dominated features. A conventional pipeline (global standardization, all features, RBF SVM) reaches only 24.7 % validation accuracy despite 99.1 % in-domain cross-validation accuracy. We show that the shift is, to first order, a per-feature affine map, which per-device standardization cancels exactly. Since test records carry no device identity, we discover devices without labels by clustering activity-independent *probe* features, and we remove noise-dominated features with a training-only ANOVA filter. The final system, an RBF SVM on 315 selected features with a numpy re-implementation of its one-vs-one decision, reaches 97.3 % validation accuracy (macro-F1 97.4 %) at 0.48 ms per sample on a CPU, all adaptation costs included. The model is trained on Device_B only; the labels of the released validation set were used for model selection, never for training.**

I. Introduction

Human activity recognition (HAR) from smartphone inertial sensors is a standard pattern recognition task: accelerometer and gyroscope windows are summarized into hand-crafted time- and frequency-domain features and classified into activities [1]. In deployment, however, the phone that produced the training data is rarely the phone that is used. Phones differ in sensor bias, sensitivity and noise, and models trained on one device degrade on others [2]. In the dataset-shift vocabulary this is a covariate shift between a source and a target domain [3], [4]; theory bounds the target error by the source error plus a divergence between the domains [5], so reducing that divergence matters as much as a strong classifier.

Problem. Given records of 561 features from Device_B (3604 labelled records), predict the activity of records from unseen devices. The validation set (3090 records) turned out to contain two devices; the hidden test set may contain others.

Related work. Domain generalization (DG) learns from source domains only, while domain adaptation also uses unlabeled target data [6], [7]. A simple and effective family re-estimates normalization statistics on the target domain: AdaBN [8] recomputes batch-normalization statistics per domain, and CORAL [9] aligns second-order statistics. Per-device calibration and normalization is also the first mitigation proposed for sensing heterogeneity in [2]. Deep models dominate HAR on raw signals [10], but on hand-crafted feature vectors kernel and linear models remain competitive [1].

TABLE I: Records per class. Device_A is empty in the release.

Activity	Train (Device_B)	Validation
0 Walking	621	526
1 Walking Upstairs	486	482
2 Walking Downstairs	494	402
3 Sitting	648	517
4 Standing	646	582
5 Laying	709	581
Total	3604	3090

Contributions. (i) A diagnosis of the released data: per-feature affine device shift, two devices in the validation set, and a noise-dominated feature block. (ii) An unsupervised device-discovery step on activity-independent probe features that enables per-device standardization at test time. (iii) A complete, tested, sub-millisecond pipeline with an honest timing protocol, evaluated by in-domain CV, simulated-shift CV and validation.

II. System Overview

A. Data and diagnosis

The training folder holds one device (Device_B; the Device_A folder of the release is empty) with the class counts of table I. Three observations drove the design (figs. 1 to 3):

1) *Calibration offsets.* 251 of 561 training features leave the $[-1, 1]$ range of the original feature definition [1], most by a constant $\approx +10$. Fitting class-conditional means of validation against training per feature gives slopes ≈ 1 and intercepts ≈ -10 on features 0–250 for the second half of the validation files: a second device with a different offset.
2) *Two validation devices.* Device discovery (below) splits the validation set into 2 groups with silhouette 0.994, exactly at file 1546.
3) *Noise-dominated features.* Features ≈40–239 have median ANOVA $F \approx$ 3–9 versus 10^2–10^4 elsewhere; a kNN classifier on these blocks is at 18.8 % accuracy even *within* Device_B (chance $= 16.7\,\%$).

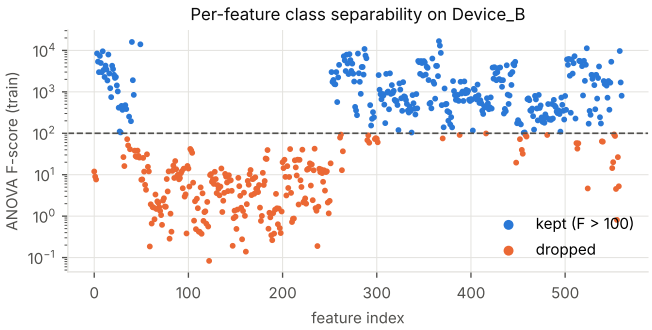


Fig. 1: Per-feature ANOVA F on Device_B (log scale) and the selection threshold.

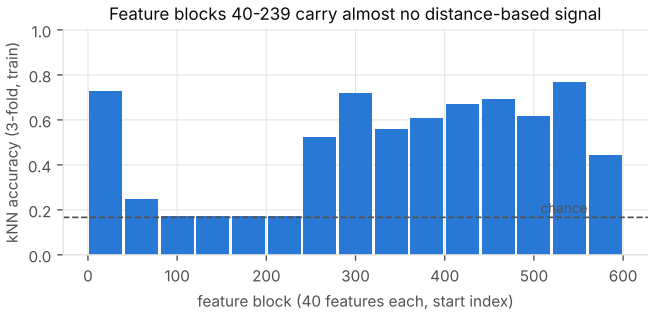


Fig. 2: kNN accuracy (3-fold, within Device_B) using one 40-feature block at a time.

B. Pipeline

Per-device standardization. A calibration offset b_j and gain $a_j > 0$ transform feature x_j into $a_j x_j + b_j$; its mean and standard deviation become $a_j\mu_j + b_j$ and $a_j\sigma_j$, so

$$\frac{(a_j x_j + b_j) - (a_j\mu_j + b_j)}{a_j\sigma_j} = \frac{x_j - \mu_j}{\sigma_j}. \qquad (1)$$

Standardizing every device with *its own* statistics therefore cancels any per-feature affine distortion; a scaler fitted on the training device cannot. Standardized values are clipped to ± 3 (winsorizing [11]; tuned in section IV) and missing values map to 0.

Unsupervised device discovery. Test files carry no device identity, and clustering on all features would recover *activities* (static vs. dynamic postures are well separated), which per-cluster standardization would erase. We therefore cluster on *probe features*: the 25 % of features with the lowest training F-score, i.e. those least related to the activity. Probes are robust-z-scored with training statistics, projected to two principal components fitted on the test set, and clustered with k-means for $k = 2 \ldots 4$. A split is accepted only if its silhouette [12] is ≥ 0.8 and every cluster holds $\geq \max(30, 5\,\% N)$ records; otherwise the test set is one device. Clusters smaller than 30 fall back to the training statistics. On each validation half alone and on the training data the rule correctly returns $k = 1$ (small outlier groups are rejected by the size rule). Figure 3 shows the two discovered devices.

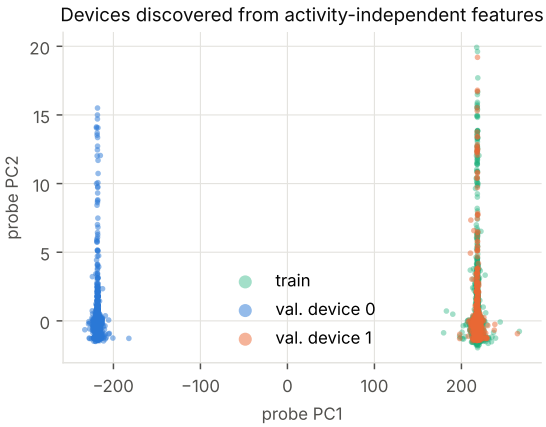


Fig. 3: Probe-feature PCA of the validation set: the discovered devices and the training data.

Feature selection. On standardized, clipped training features we keep those with ANOVA $F > 100$ (a univariate filter [13]), 315 of 561. The threshold is not fragile: validation accuracy is flat for thresholds between 25 and 200 (fig. 6).

Classifier. An RBF SVM [14] with $C = 100$ and γ=`scale` (one-vs-one [15], libsvm [16]); 1059 support vectors; model file 2.8 MB.

Inference and timing. `infer.py` lists `1.csv...N.csv` numerically, reads all files (untimed I/O, as the brief specifies), runs the one-off adaptation (device discovery + statistics + one warm-up prediction, 0.34 s on the validation set), then times each sample's standardization and prediction individually. Each line of `time.txt` is that time plus the adaptation time divided by N, so no computation is hidden. For speed the SVM decision is re-implemented in numpy: one matrix–vector product gives all kernel values, a precomputed $15 \times n_{SV}$ matrix gives the 15 pairwise decisions, and a vote picks the class. It is verified identical to scikit-learn [17] at training time and in the tests (byte-identical `results.txt` on all validation files) and cuts per-sample latency by $\approx 3\times$.

Engineering. The code is one Python package with a module per stage (data, preprocessing, features, models, evaluation, inference, experiments), strict type checking, 56 unit/integration tests on synthetic data, import-boundary contracts, continuous integration that also builds the Docker image and runs it, and a Gradio demo for Hugging Face Spaces (*TODO(team): Hugging Face Space URL*). Source: https://github.com/3essawyy/NNP.

III. Experimental Setup

Data usage. Training uses the Device_B folder only. The labels of the released validation set are used for model selection and hyperparameter tuning, never for fitting. Because they guide selection, the validation score is mildly optimistic; we therefore also report two estimates that do not touch validation labels.

Three estimates. (1) *In-domain CV*: stratified 5-fold CV on Device_B (upper bound, no shift). (2) *Simulated-shift CV*: same folds, but each held-out fold is turned into a synthetic new phone before prediction: offsets of $\mathcal{N}(0, 3\sigma_j)$ on a random half of the features, log-normal gains ($\sigma = 0.2$), per-record heteroscedastic noise ($0.1\sigma_j$ scaled by $U(0.5, 1.5)$) and 1 % outlier records. The pipeline must adapt to the fold on its own. (3) *Validation*: the released set, overall and per discovered device. No subject identifiers exist, so subject-wise CV is impossible; with a second labelled device, leave-one-device-out CV would replace (2).

Preprocessing and features. Per-device z-score (robust median/IQR scaling compared), clipping at ± 3, ANOVA F threshold; alternatives in the ablation: no adaptation, no selection, robust scaling, mutual-information top-k, PCA (95 % variance), LDA projection.

Models. Gaussian NB, LDA with Ledoit–Wolf shrinkage [18], regularized QDA, logistic regression, linear SVM, RBF SVM, kNN ($k = 10$, distance-weighted), random forest and extra trees (300 trees), LightGBM [19] (300 rounds), an MLP (256 hidden units), and a hard-voting ensemble (RBF SVM, linear SVM, logistic regression), all with the same preprocessing.

Tuning. Grid $C \in \{1, 3, 10, 30, 100\} \times F_{\text{thr}} \in \{10, 25, 50, 100, 200, 400\}$, then $\gamma \in \{0.25, 0.5, 1, 2, 4\} \times \gamma_{\text{scale}}$, then clip $\in \{2, 3, 4, 5, 8\} \times$ {z-score, robust}; the $C \times F$ grid was re-run at the chosen clip to check consistency. Rule: highest validation accuracy; ties within 0.2 pp go to the higher simulated-shift CV, then to the simpler model.

Metrics. Accuracy (the leaderboard metric), macro-F1, per-class precision/recall, confusion matrices, and per-sample time as written to `time.txt`. Hardware: 4 vCPU cloud container, Python 3.12, scikit-learn 1.9, single-threaded BLAS.

IV. Results

A. Model comparison

Table II and fig. 4 compare all families under the shipped preprocessing, each with default hyperparameters. Per-device adaptation improves every family, by 7.5 pp (MLP) to 36 pp (LightGBM). With a global scaler, ± 3 clipping already limits how far a calibration offset can push a standardized feature, so some models keep 80–88 %, but none comes close to the adapted pipelines. The RBF SVM is the best family on all three estimates (96.7 % on validation at its default $C = 10$) and costs about half a millisecond per sample. A hard-voting ensemble of the three strongest models is *not* better (96.4 %) and is three times slower. Tree ensembles transfer worse and cost ≈88 ms per single-row prediction, two orders of magnitude more than the SVM, which matters because running time enters the ranking. Simulated-shift CV ranks the families much like validation does (4 of the top 5 agree).

B. Ablation

Table III and fig. 5 remove one step at a time from the shipped pipeline (97.3 %). Without per-device adaptation accuracy falls to 75.9 %; without feature selection to 30.1 %; without both to 24.7 %, although that conventional pipeline scores 99.1 % in in-domain cross-validation. Only the combination works. After per-device z-scoring, the ≈246 noise-dominated dimensions have the same unit variance as the informative ones and dominate RBF distances, so removing the offsets helps only once those dimensions are dropped. Mutual information is not a substitute for the ANOVA filter. Its top 315 features include 32 noise-block features, and accuracy drops to 81 %. MI detects non-linear dependence that survives the noise but does not transfer. Clipping adds 0.4 pp, robust (median/IQR) scaling is 0.6 pp lower, and neither PCA nor an LDA projection helps (the latter collapses under simulated shift).

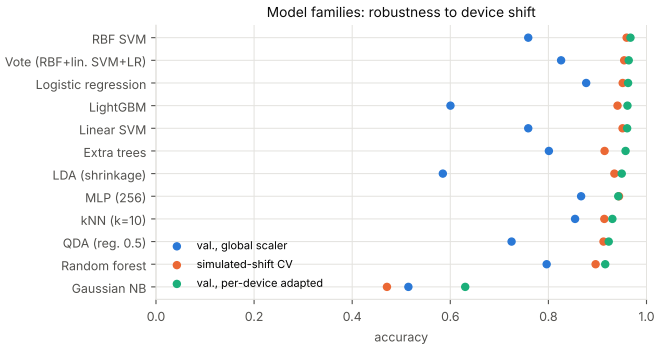


Fig. 4: Accuracy of each family: validation with a global scaler, simulated-shift CV, and validation with per-device adaptation.

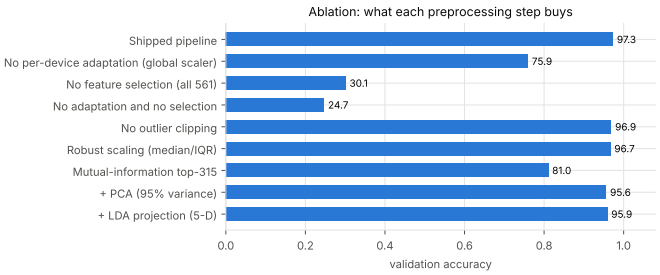


Fig. 5: Ablation of the preprocessing (validation accuracy).

A blind spot of simulated-shift CV. The synthetic phone rates the pipeline *without* feature selection at 98 %, the best variant, while validation gives 30 %. Our synthetic shift (offsets, gains, noise) does not reproduce what makes the noise block non-transferable. We therefore treat simulated-shift CV as evidence about calibration robustness only; the feature-selection decision rests on the training-only diagnosis (fig. 2) and on validation.

C. Tuning

Figure 6 shows the $C \times F_{\text{thr}}$ grid at the final clip of ± 3 (range 84.7–97.3 %). A threshold of 10 keeps part of the noise block (85–88 %). For thresholds of 25–400 and $C \geq 3$ the surface is a broad plateau of 95.7–97.3 %, so the chosen point ($C = 100$, $F > 100$, 97.3 %) is not a sharp, validation-specific optimum.

TABLE II: Model families with the same preprocessing (ANOVA $F > 100$, clip ± 3). Accuracy in %; 5-fold CV on Device_B; validation = released eval_data (never trained on).

Model	In-domain CV	Sim.-shift CV	Val. (global scaler)	Val. (adapted)	ms/sample
RBF SVM	96.3 ± 0.8	95.9	75.9	**96.7**	0.51
Vote (RBF+lin. SVM+LR)	95.9 ± 0.9	95.4	82.6	96.4	1.47
Logistic regression	95.9 ± 0.8	95.1	87.7	96.2	0.30
LightGBM	95.6 ± 0.7	94.1	60.0	96.1	1.43
Linear SVM	95.6 ± 0.9	95.1	75.9	96.1	0.34
Extra trees	94.2 ± 1.6	91.5	80.1	95.7	87.82
LDA (shrinkage)	94.5 ± 1.0	93.5	58.5	95.0	0.29
MLP (256)	94.7 ± 1.9	94.4	86.7	94.2	0.53
kNN (k=10)	91.6 ± 1.1	91.4	85.4	93.0	1.26
QDA (reg. 0.5)	90.9 ± 1.6	91.2	72.5	92.3	1.54
Random forest	92.6 ± 1.8	89.6	79.6	91.6	87.53
Gaussian NB	68.2 ± 0.5	47.1	51.5	63.0	0.65

TABLE III: Ablation with the shipped RBF SVM (accuracy %). Per-device columns use the devices discovered on the validation set; variants without adaptation treat the set as one domain (–).

Variant	Val. dev. 0	Val. dev. 1	Val. overall	Sim.-shift CV
Shipped pipeline	97.2	97.4	97.3	95.6
No per-device adaptation (global scaler)	–	–	75.9	18.4
No feature selection (all 561)	31.1	29.1	30.1	98.2
No adaptation and no selection	–	–	24.7	19.7
No outlier clipping	97.0	96.8	96.9	94.4
Robust scaling (median/IQR)	96.2	97.1	96.7	95.4
Mutual-information top-315	81.4	80.6	81.0	96.2
+ PCA (95% variance)	95.1	96.1	95.6	93.8
+ LDA projection (5-D)	95.6	96.2	95.9	58.2

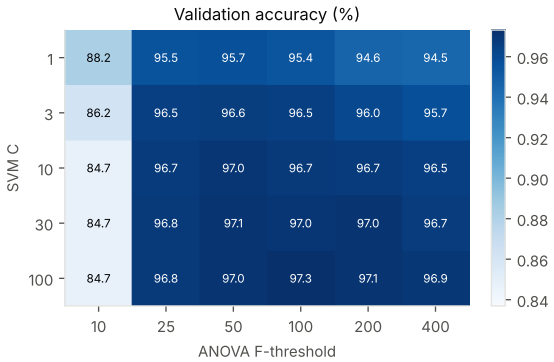


Fig. 6: Validation accuracy over SVM C and ANOVA threshold (clip ± 3).

The default γ is the peak of a γ sweep. Table IV tunes the outlier clip and the scaling estimator. Clipping at ± 2–± 5 beats ± 8, and z-scoring beats robust scaling by 0.5–0.8 pp; the rule picks ± 3. Re-running the grid at ± 3 re-selects the same C and threshold. Against the untuned default ($C = 10$, $F > 50$, ± 8: 96.7 %), tuning gains 0.6 pp, about 19 files. That is of the order of the optimism introduced by selecting on validation.

D. Robustness to controlled shifts

Figure 7 applies increasing synthetic shifts to a held-out 20 % of Device_B. Per-device adaptation makes accuracy exactly invariant to calibration offsets and gains, as the algebra in section II predicts. The global scaler collapses once offsets reach two standard deviations and falls below chance by ten. Additive noise degrades both (adaptation removes no noise). Outlier *records* are a genuine weakness of adaptation: they contaminate the device's own mean and standard deviation, so at 20 % outliers the adapted pipeline keeps 85 % while the global scaler, whose statistics come from clean training data, keeps 95 %. Clipping at ± 3 recovered part of this (from 77 % at ± 8); robust statistics would recover more at a small validation cost.

TABLE IV: Outlier clipping and scaling at $C = 100$, $F > 100$ (accuracy %; Val. = validation, Sim. = simulated-shift CV; z = z-score, robust = median/IQR).

Clip	Val. z	Val. robust	Sim. z	Sim. robust
± 2	97.38	96.57	96.14	94.48
± 3	97.31	96.67	96.20	94.67
± 4	97.25	96.73	95.95	94.92
± 5	97.25	96.76	96.00	94.62
± 8	97.06	96.60	95.53	93.95

E. Small test folders

Adaptation needs a batch. Figure 8 evaluates random subsets of the validation set with $N = 1 \ldots 200$ files, drawn from both devices or from one. With the S002 settings (minimum

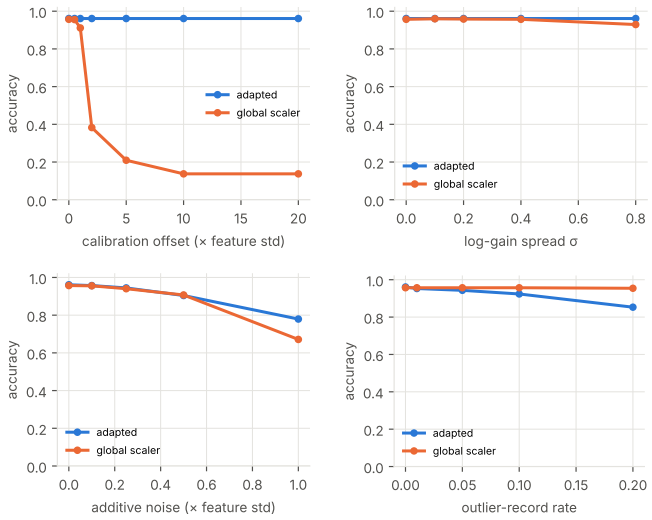


Fig. 7: Accuracy under controlled synthetic device shifts: adapted vs. global scaler.

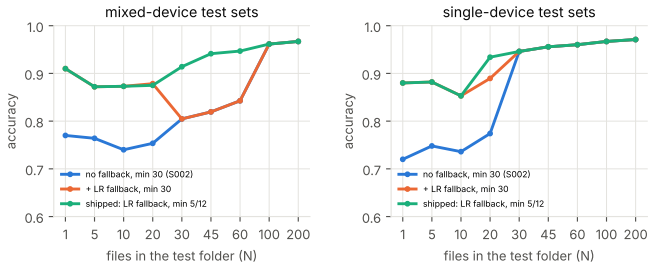


Fig. 8: Accuracy on random validation subsets of N files (mixed or single device).

30 records per discovered device), mixed folders of 30 files reached only 80.5 %. They could not be split, so one set of statistics was computed over two phones. Folders under 30 files fell back to training statistics and the RBF SVM. The shipped settings split devices from 5 records per cluster and use a device's own statistics from 12. Records that still fall back go to a logistic regression trained on training-standardized features, which tolerates residual offsets far better than the RBF kernel (87.7 % vs. 75.9 % per file). This raises mixed 30-file folders to 91.4 % and single-device 20-file folders from 77.4 % to 93.4 %; a single file scores 91.0 %. The full validation score is unchanged. Folders of 10–20 files remain the weakest case (86–88 %).

F. Final system and error analysis

Through the exact `infer.py` code path, the shipped model reaches 97.3 % accuracy (macro-F1 97.4 %) on the validation set: 97.2 % and 97.4 % on the two discovered devices (table V and fig. 9). Of the 83 errors, 72 are Sitting ↔ Standing. Both are static upright postures with nearly identical inertial signatures, the classic confusion of this feature set [1]. The remaining 11 errors are among the three walking classes. No

TABLE V: Shipped model on the validation set, through the real `infer.py` path (%).

Activity	Precision	Recall	F1	Support
0 Walking	99.2	99.2	99.2	526
1 Walking Upstairs	99.6	99.0	99.3	482
2 Walking Downstairs	98.8	99.5	99.1	402
3 Sitting	91.9	94.4	93.1	517
4 Standing	94.9	92.6	93.7	582
5 Laying	100.0	100.0	100.0	581

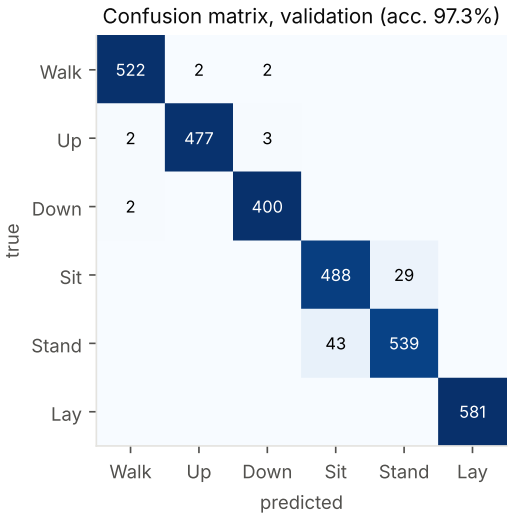


Fig. 9: Confusion matrix of the shipped model on the validation set (counts; colour = row-normalized).

dynamic activity is ever confused with a static one, and Laying is recognized perfectly thanks to its distinct gravity orientation. The failure mode is the same on both devices, so it stems from the features, not from the shift.

Timing: mean 0.48 ms and 95th percentile 0.65 ms per sample as written to `time.txt`, 1.49 s in total for 3090 samples, including the amortized 0.34 s adaptation (fig. 10).

V. Conclusion

Capabilities. The system recognizes six activities on unseen phones with 97.3 % validation accuracy at 0.48 ms per sample on a CPU. It does so by turning the device-shift problem into a normalization problem: discover devices without labels, then standardize each with its own statistics. The pipeline is reproducible end to end (`poe train`, `poe experiments`, Docker) and every report number is generated from code.

Limitations. (i) The correction is first-order: it cancels per-feature offsets and gains, not nonlinear distortions or cross-feature mixing. (ii) Per-device statistics assume similar activity mixes across devices; a device dominated by one activity would be mis-centred (prior shift). (iii) Adaptation needs a batch: device groups under 12 records fall back to training statistics and a logistic-regression fallback (about 88 % per file), and folders of 10–20 files remain the weakest case (86–88 %); and outlier *records* contaminate a device's statistics

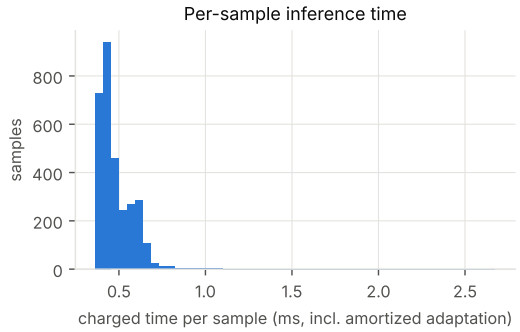


Fig. 10: Per-sample time charged in `time.txt`.

(85 % vs. 95 % for a global scaler at 20 % outliers). (iv) Only one labelled training device was available, and validation labels guided model selection, so the validation score is optimistic for truly new phones. (v) Sitting and Standing remain confusable.

Future work. Robust or trimmed per-device statistics against outlier records; second-order alignment (CORAL [9]) or class-balanced statistics estimated with pseudo-labels to address prior shift; leave-one-device-out training once Device_A data is available; a posture-specific second-stage classifier for Sitting/Standing; and per-sample adaptation for streaming use.

VI. Workload Distribution

Member	Contributions
Member 1	Problem understanding & system design; data module; inference script and timing protocol; Docker; Hugging Face deployment
Member 2	Preprocessing & feature engineering: shift diagnosis, per-device standardization, device discovery, feature selection
Member 3	Model selection & training: model zoo, cross-validation protocols, hyperparameter tuning, fast SVM inference
Member 4	Experimental evaluation & analysis: metrics, ablation, robustness study, error analysis; report

References

[1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, 2013.

[2] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proc. 13th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2015.

[3] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. MIT Press, 2009.

[4] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, "A unifying view on dataset shift in classification," *Pattern Recognition*, vol. 45, no. 1, 2012.

[5] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Machine Learning*, vol. 79, 2010.

[6] K. Zhou, Z. Liu, Y. Qiao, T. Xiang, and C. C. Loy, "Domain generalization: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[7] J. Wang, C. Lan, C. Liu, Y. Ouyang, T. Qin, W. Lu, Y. Chen, W. Zeng, and P. S. Yu, "Generalizing to unseen domains: A survey on domain generalization," *IEEE Transactions on Knowledge and Data Engineering*, 2022.

[8] Y. Li, N. Wang, J. Shi, J. Liu, and X. Hou, "Revisiting batch normalization for practical domain adaptation," in *International Conference on Learning Representations (ICLR) Workshop*, 2017, arXiv:1603.04779.

[9] B. Sun, J. Feng, and K. Saenko, "Return of frustratingly easy domain adaptation," in *Proc. AAAI Conference on Artificial Intelligence*, 2016.

[10] J. Wang, Y. Chen, S. Hao, X. Peng, and L. Hu, "Deep learning for sensor-based activity recognition: A survey," *Pattern Recognition Letters*, vol. 119, 2019.

[11] P. J. Huber, *Robust Statistics*. Wiley, 1981.

[12] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, vol. 20, 1987.

[13] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, 2003.

[14] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, 1995.

[15] C.-W. Hsu and C.-J. Lin, "A comparison of methods for multiclass support vector machines," *IEEE Transactions on Neural Networks*, vol. 13, no. 2, 2002.

[16] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, 2011.

[17] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.

[18] O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, 2004.

[19] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.